

PROGRAMACIÓN CONCURRENTE Y PARALELA

Práctica opcional: Haskell, IO y mónadas

2do Cuatrimestre 2026

Esta guía es opcional. Es un warm-up para la Práctica 5. Los ejercicios se pueden resolver en `ghci` o compilando con `ghc`.

Warm-up

Ejercicio 1. Funciones sobre listas. Definir de manera recursiva, sin usar las funciones equivalentes del *Prelude*:

- a) `longitud :: [a] -> Int`
- b) `sumatoria :: [Int] -> Int`
- c) `pertenece :: Eq a => a -> [a] -> Bool`
- d) `quitar :: Eq a => a -> [a] -> [a]`, que quita solo la primera aparición del elemento.

Ejercicio 2. Funciones de alto orden.

- a) Definir `cuadrados :: [Int] -> [Int]` usando `map` y `positivos :: [Int] -> [Int]` usando `filter`.
- b) Definir `cantidad :: (a -> Bool) -> [a] -> Int` que cuenta cuántos elementos cumplen el predicado, usando `foldr`.

Ejercicio 3. Considerar el siguiente tipo para árboles binarios de búsqueda:

```
data Arbol a = Hoja | Nodo (Arbol a) a (Arbol a)
```

Definir:

- a) `insertar :: Ord a => a -> Arbol a -> Arbol a`, que no agrega elementos repetidos.
- b) `estaEn :: Ord a => a -> Arbol a -> Bool`
- c) `enOrden :: Arbol a -> [a]`, que devuelve los elementos ordenados de menor a mayor.
- d) `desdeLista :: Ord a => [a] -> Arbol a`, que construye un árbol insertando todos los elementos de la lista. Usar `foldr`.

Ejercicio 4. Funciones parciales con `Maybe`.

- a) `safeHead :: [a] -> Maybe a`, que devuelve el primer elemento de la lista, o `Nothing` si la lista es vacía.
- b) `buscar :: Eq k => k -> [(k, v)] -> Maybe v`, que devuelve el valor asociado a la clave, o `Nothing` si la clave no está.
- c) `dividir :: Int -> Int -> Maybe Int`, que devuelve `Nothing` si el divisor es 0.
- d) Dado el tipo

```
data Cuenta = Cuenta { titular :: String, saldo :: Int }
```

definir las siguientes funciones. **extraer** devuelve **Nothing** si el saldo no alcanza.

```
depositar :: Int -> Cuenta -> Cuenta
extraer   :: Int -> Cuenta -> Maybe Cuenta
```

Mónadas

Ejercicio 5. **Maybe** y **Either** como mónadas.

- Definir `calcular :: Int -> Int -> Int -> Maybe Int` tal que `calcular a b c` devuelve $(a / b) / c + 1$ usando `dividir` del Ejercicio 4. Dar tres versiones: con **case**, con `>>=` y con **do**.

- Usando la notación **do** y las funciones del Ejercicio 4, definir

```
transferir :: Int -> Cuenta -> Cuenta -> Maybe (Cuenta, Cuenta)
```

que devuelve las cuentas de origen y destino luego de transferir el monto, o **Nothing** si el saldo de origen no alcanza.

- Dar versiones de `dividir`, `extraer` y `transferir` que devuelvan **Either String** con un mensaje que explique el error. ¿Cuánto cambia el código de `transferir`?

Ejercicio 6. La lista como mónada.

- Definir con notación **do** la función `pares :: [a] -> [b] -> [(a, b)]` que devuelve todos los pares posibles.
- Definir con notación **do** la función `pitagoricas :: Int -> [(Int, Int, Int)]` que devuelve las ternas (a, b, c) con $a \leq b \leq c \leq n$ y $a^2 + b^2 = c^2$, sin usar listas por comprensión.

Ejercicio 7. Las siguientes funciones de `Control.Monad` sirven para cualquier mónada. Definirlas sin usar las de la biblioteca, y probarlas con **Maybe** y listas.

- `when :: Monad m => Bool -> m () -> m ()`, que ejecuta la acción solo si la condición es verdadera. Si es falsa, no hace nada.
- `replicateM_ :: Monad m => Int -> m a -> m ()`, que ejecuta la acción la cantidad de veces indicada, una después de la otra, y descarta los resultados.
- `mapM :: Monad m => (a -> m b) -> [a] -> m [b]`, que aplica la función a cada elemento de la lista, ejecuta las acciones en orden y devuelve la lista de resultados. ¿Qué devuelve `mapM (dividir 100) [1, 0, 5]`?
- `forM_ :: Monad m => [a] -> (a -> m b) -> m ()`, que es como `mapM` con los argumentos invertidos, pero descarta los resultados.
- `forever :: Monad m => m a -> m b`, que ejecuta la acción una y otra vez, sin terminar. ¿Qué devuelve `forever Nothing`?

Entrada y salida

Ejercicio 8. Programas interactivos.

- Escribir un programa que pregunte el nombre del usuario y lo salude por su nombre.
- Definir `sumarHastaCero :: IO Int`, que lee números de la entrada (uno por línea) hasta leer un 0 y devuelve la suma de los números leídos. Escribir un `main` que la use e imprima el resultado.

Ejercicio 9. Una expresión de tipo `IO a` es una *acción*: un valor que describe un efecto, que se ejecuta solo cuando forma parte de `main`. Sea

```
hola :: IO ()
hola = putStrLn "hola"
```

- Indicar qué imprime cada uno de los siguientes programas y justificar.

```
programa1 = do
    let x = hola
    putStrLn "chau"

programa2 = do
    let x = hola
    x
    x

programa3 = do
    n <- return 5
    return 10
    print n

programa4 = do
    let as = [putStrLn "a",
              putStrLn "b",
              putStrLn "c"]

    as !! 2
    head as
```

- Definir `repetir :: Int -> IO () -> IO ()` que ejecuta una acción la cantidad de veces indicada.

Ejercicio 10. Definir `adivinar :: Int -> IO ()`, que recibe un número secreto y le pide al usuario que lo adivine. Después de cada intento, el programa le dice si el número secreto es más alto o más bajo. Cuando el usuario acierta, el programa muestra cuántos intentos le llevó y termina. Escribir un `main` que la use.